

Lab 4 Task

Q1. Read the following C program carefully. Your task is to dry run the program **on paper** and answer the questions below:

```
#include <stdio.h>
#include <unistd.h>

int main() {
    int a = 1;

    fork();
    a += 1;

    fork();
    a += 2;

    printf("PID: %d, a = %d\n", getpid(), a);

    return 0;
}
```

1. How many total processes are created **after both fork() calls**?
2. How many times will the printf() statement be executed? Explain why.
3. What is the range of possible values for variable **a** printed by different processes?
4. Which process creates which process? (Draw a process tree)
5. Can we predict the **order** of the output? Why or why not?

Q2. Carefully read the following program and answer the questions. **You are not expected to dry run it line by line on paper.** Instead, you should run it on a Linux terminal and observe the behavior.

```
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("Before replacing the process\n");

    execl("/bin/echo", "echo", "Hello from exec!", NULL);

    printf("After exec - will this line run?\n");

    return 0;
}
```

1. What is the role of `execl()` in this program?
2. Which line(s) will actually be printed when this program runs?
3. Why does the second `printf()` line not run (if it doesn't)?
4. What would happen if the path `/bin/echo` was incorrect?
5. Try removing the `execl()` call — what changes in the program's behavior?

Q3. Run this code **once with `wait()`** and **once after commenting `wait(NULL)`**; Observe the difference in output order

Write 1-line comment: What changed when `wait()` was removed?

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pid = fork(); // Create new process

    if (pid == 0) {
        // Child process
        printf("Child Process:\n");
        printf("PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());
        sleep(2); // Sleep for 2 seconds
        printf("Child finished.\n");
    } else {
        // Parent process
        wait(NULL); // Comment this line to test "without wait"
        printf("Parent Process:\n");
        printf("PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
        printf("Parent finished.\n");
    }

    return 0;
}
```